

INTRODUÇÃO

1. O que é Heap?

Heap é uma estrutura de dados baseada em árvore binária que segue regras específicas de organização.

Ela é chamada de árvore binária completa, porque todos os níveis da árvore são preenchidos da esquerda para a direita.

A principal característica do Heap é a chamada:

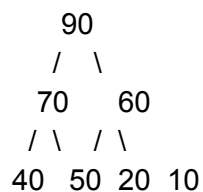
Propriedade Heap

Existem dois tipos principais:

Max-Heap

O valor do nó pai é maior ou igual aos valores dos filhos.

Exemplo:

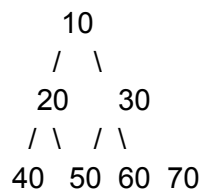


O maior elemento sempre fica na raiz.

Min-Heap

O valor do nó pai é menor ou igual aos valores dos filhos.

Exemplo:



O menor elemento sempre fica na raiz.

Características do Heap

- Estrutura em forma de árvore binária.
- Árvore completa.
- Inserção e remoção eficientes.
- Muito usado em algoritmos de prioridade.
- Normalmente implementado usando vetor/array.

2. Para que serve o Heap?

O Heap serve para organizar dados de maneira eficiente quando precisamos acessar rapidamente:

- O maior elemento (Max-Heap)
- O menor elemento (Min-Heap)

Principais objetivos do Heap

1. Filas de prioridade

O Heap é muito usado em filas de prioridade.

Exemplo:

- Hospital → pacientes mais graves primeiro.
- Jogos → jogadores com maior pontuação.
- Sistemas operacionais → processos prioritários.

2. Ordenação de dados (HeapSort)

Heap é usado no algoritmo HeapSort.

O HeapSort:

- Organiza elementos usando Heap.
- Possui complexidade $O(n \log n)$.
- É eficiente para grandes volumes de dados.

3. Busca rápida do maior ou menor elemento

Como o maior ou menor valor fica na raiz:

- acesso é rápido;
- remoção é eficiente.

4. Sistemas em tempo real

Usado em:

- escalonamento de tarefas;
- gerenciamento de memória;
- inteligência artificial;
- redes.

3. Onde o Heap é usado?

Heap aparece em várias áreas da computação.

Sistemas Operacionais

Usado no:

- escalonamento de processos;
- controle de prioridades.

Exemplo:

O sistema decide qual processo será executado primeiro.

Inteligência Artificial

Usado em:

- algoritmos de busca;
- processamento de decisões;
- jogos.

Algoritmos de Grafos

Muito usado no algoritmo de Dijkstra.

O Heap ajuda a encontrar:

- o menor caminho;
- vértices prioritários.

Bancos de Dados

Utilizado para:

- ordenação;
- consultas rápidas;
- gerenciamento de dados.

Jogos

Usado em:

- ranking;
- matchmaking;

- sistemas de pontuação.

Redes de Computadores

Usado em:

- roteamento;
- controle de pacotes;
- prioridade de tráfego.

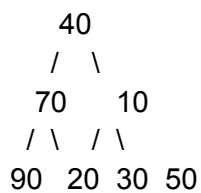
4. Diferença entre árvore normal e Heap

Árvore Binária Normal

Uma árvore binária comum:

- não possui regra obrigatória entre pai e filhos;
- pode ter qualquer organização.

Exemplo:



Não existe relação de ordem.

Heap

No Heap existe uma regra obrigatória:

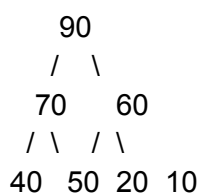
Max-Heap

Pai $\geq$ filhos

Min-Heap

Pai $\leq$ filhos

Exemplo de Max-Heap:



Comparação direta

Característica	Árvore Binária Normal	Heap
Organização	Livre	Segue regra
Estrutura completa	Nem sempre	Sempre
Elemento maior/menor	Não garantido	Fica na raiz
Inserção	Variável	Eficiente
Uso principal	Estruturas gerais	Prioridade e ordenação

Vantagens do Heap

- Inserção rápida.
- Remoção eficiente.
- Fácil acesso ao maior/menor elemento.
- Muito eficiente em filas de prioridade.
- Excelente para grandes volumes de dados.

Desvantagens do Heap

- Não mantém todos os elementos totalmente ordenados.
- Busca de elemento específico pode ser lenta.
- Implementação pode ser mais complexa para iniciantes.

REFERÊNCIAS

LEITE, Thiago et al. **Estruturas de Dados: Domine as práticas essenciais em C, Java, C#, Python e JavaScript**. Casa do Código, 2023.

DE MORAES, Pedro Henrique Sousa. Willow: Uma Ferramenta com Conceitos de Programação Interativa para Auxiliar no Ensino de Introdução a Programação e Algoritmos e Estruturas de Dados.

SUAVE, André Augusto. **Estrutura de Dados**. Freitas Bastos, 2025.

